

ตัวอย่างลำดับการประมวลผลของฟังก์ชัน setup และ loop

```
#include <POP32.h>
```

```
void setup()
```

```
{
```

```
  command1;
```

```
  command2;
```

```
  command3;
```

```
}
```

```
void loop()
```

```
{
```

```
  command4;
```

```
  command5;
```

```
  command6;
```

```
}
```

การประมวลผลโปรแกรมจะทำงานดังนี้

```
command1;
```

```
command2;
```

```
command3;
```

```
command4;
```

```
command5;
```

```
command6;
```

```
command4;
```

```
command5;
```

```
command6;
```

```
command4;
```

```
command5;
```

```
command6;
```

```
...
```

```
...
```

```
...
```

รอบเดียวสำหรับ setup

รอบที่ 1 สำหรับ loop

รอบที่ 2 สำหรับ loop

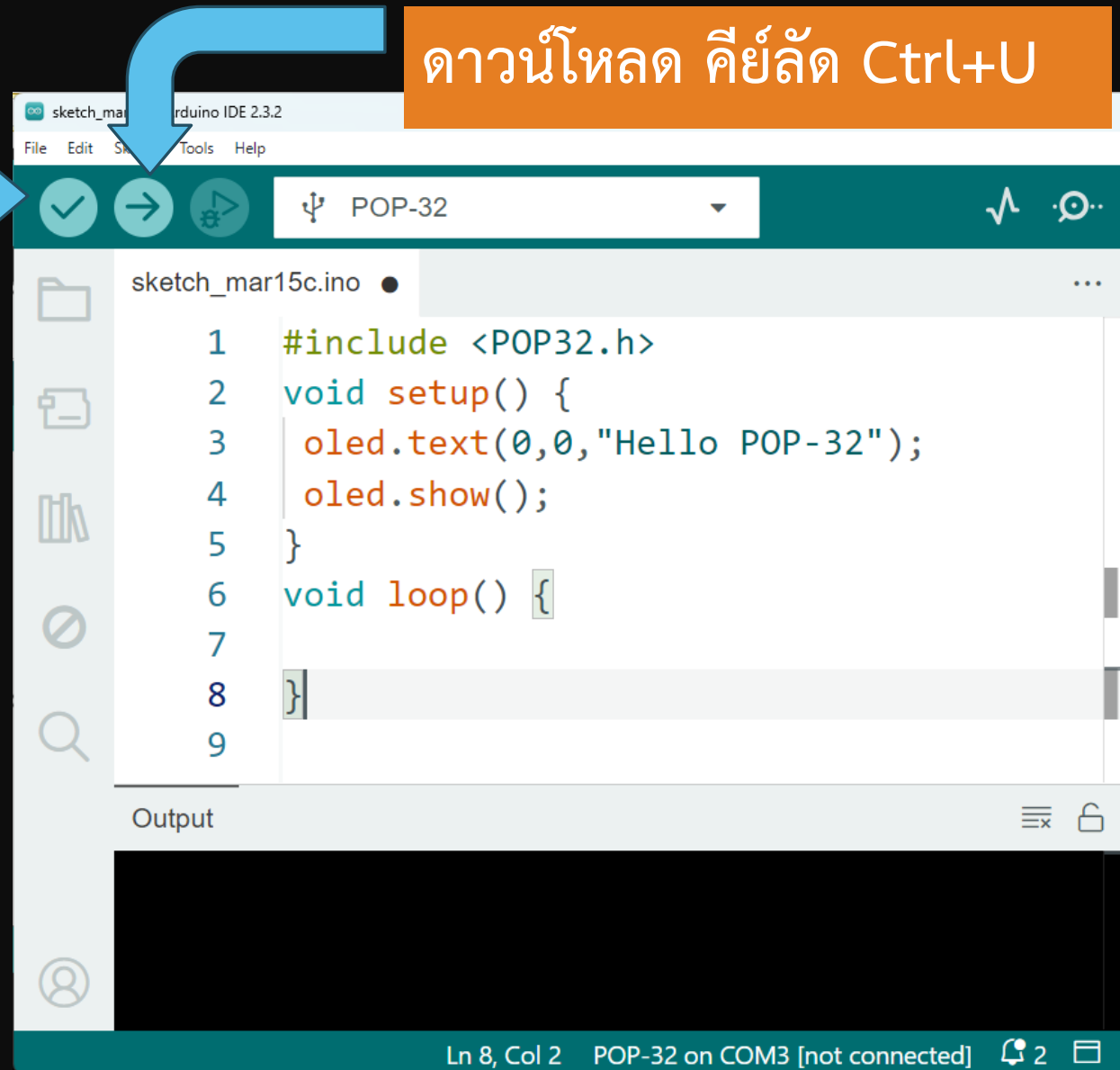
รอบที่ 3 สำหรับ loop

รอบที่ n สำหรับ loop

เขียนโปรแกรมผ่าน Arduino

คอมไพล์ คีย์ลัด Ctrl+R

ดาวน์โหลด คีย์ลัด Ctrl+U



แจ้งผลการคอมไพล์และขนาดหน่วยความจำที่ใช้

อัปโหลดโค้ดไปยัง POP-32
คีย์ลัด Ctrl+U

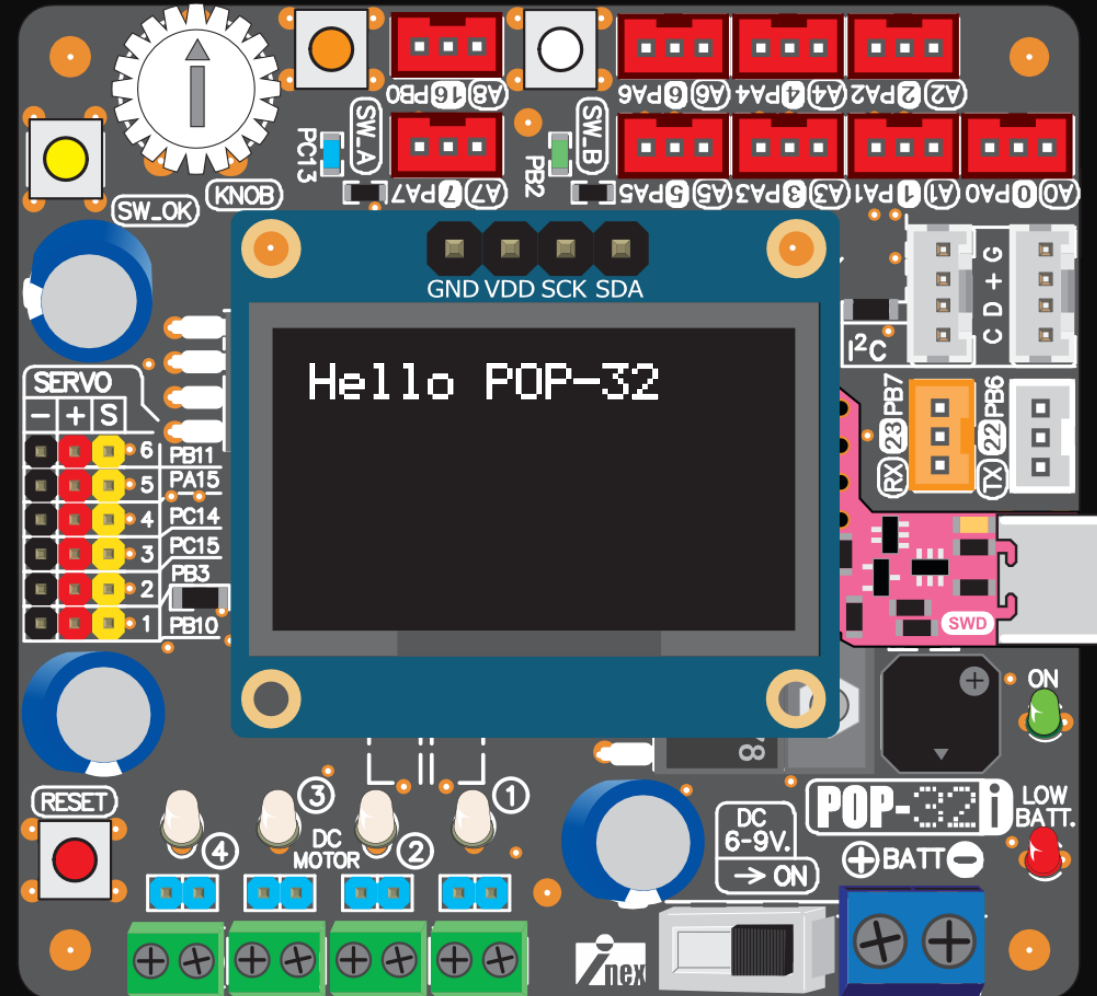


แจ้งผลว่าสมบูรณ์

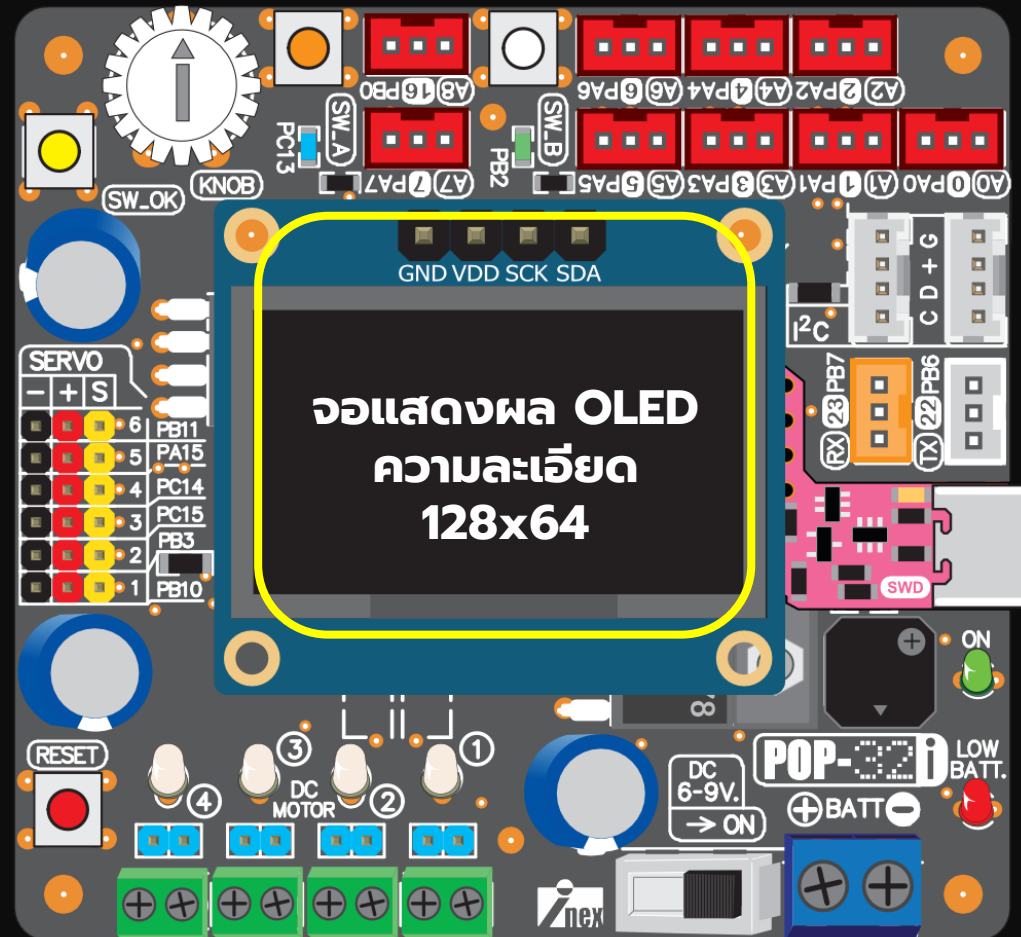
```
Output
```

```
RUNNING Program ...  
| Address:      : 0x8000000  
Application is running, Please Hold on...  
Start operation achieved successfully
```

Ln 6, Col 14 POP-32 on COM3 [not connected] 4



กลุ่มคำสั่งแสดงผลที่หน้าจอ OLED



ชุดคำสั่งสำหรับควบคุมการแสดงผล

คลาส OLED_I2C_SSD1309 เป็นคลาสที่รวบรวมเมธอดต่างๆ สำหรับควบคุมการแสดงผลกราฟิกแบบ OLED ความละเอียด 128x64 พิกเซล อย่างไรก็ดีเพื่อความสะดวกในการควบคุมการแสดงผลกราฟิกภายในคลาส OLED_I2C_SSD1309 ได้ทำการประกาศสร้างออบเจกต์ที่ทำการสืบทอดมาจากคลาส OLED_I2C_SSD1309 โดยตั้งชื่อเป็น **oled** เป็นที่เรียบร้อยแล้ว

คำสั่งสำหรับควบคุมการแสดงผลข้อความ

oled.text แสดงข้อความที่จอแสดงผล OLED ได้ 21 ตัว 8 บรรทัด (textSize=1)

รูปแบบ

```
oled.text(x,y,*p,...);
```

พารามิเตอร์

x คือตำแหน่งบรรทัดมีค่าตั้งแต่ 0-7

y คือตำแหน่งตัวอักษรมีค่าตั้งแต่ 0-20

***p** คือข้อความที่ต้องการนำมาแสดง

ค่าพิเศษ

%d แสดงตัวเลขจำนวนเต็มในช่วง -2,147,483,648 ถึง 2,147,483,647

%h แสดงตัวเลขฐานสิบหก

%b แสดงตัวเลขฐานสอง

%f แสดงผลตัวเลขจำนวนจริง (แสดงทศนิยม 3 หลัก)

คำสั่งสำหรับควบคุมการแสดงผลข้อความ

`oled.show`

ทำหน้าที่อัปเดตการแสดงผล

รูปแบบ

`oled.show()`

พารามิเตอร์ ไม่มี

การคืนค่า ไม่มี

รูปแบบ คำสั่ง delay

delay หน่วงเวลาในหน่วยมิลลิวินาที
รูปแบบ

delay(time)

พารามิเตอร์

time คือ เวลาที่ต้องการหน่วงในหน่วยมิลลิวินาที

ตัวอย่างเช่น

delay(500); // หน่วงเวลา 500 มิลลิวินาที(0.5 วินาที)

delay(2000); // หน่วงเวลา 2000 มิลลิวินาที(2 วินาที)

ตัวแปร (ที่ใช้งานบ่อย)

byte	0-255	(unsigned char)
bool	0-1	true false
int	-2147483648 ถึง 2147483647	
char	-128 ถึง 127	
float	-3.4×10^{38} ถึง 3.4×10^{38}	

หาข้อมูลเพิ่มเติมจาก [reference](#)

ชุดคำสั่งอื่นๆ สำหรับใช้งานอ็อบเจกต์ oled

แสดงตัวอักษร

text
textSize
mode
textColor
textBackgroundColor
clear
fillScreen

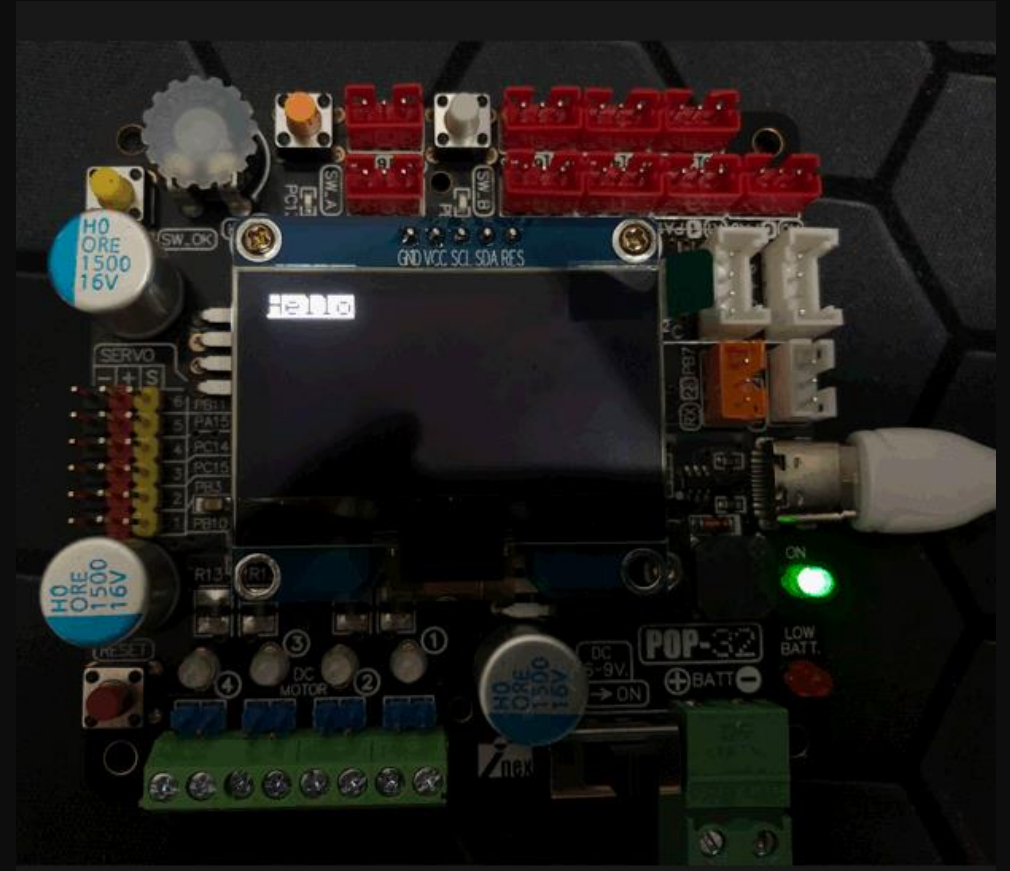
แสดงกราฟิก

drawPixel
drawRect
fillRect
drawLine
drawCircle
fillCircle

ตัวอย่างกำหนดสีตัวอักษร

```
#include <POP32.h>
void setup(){
  oled.setTextColor(BLACK,WHITE);
  oled.text(0,0,"Hello world");
  oled.show();
}
void loop(){}

```

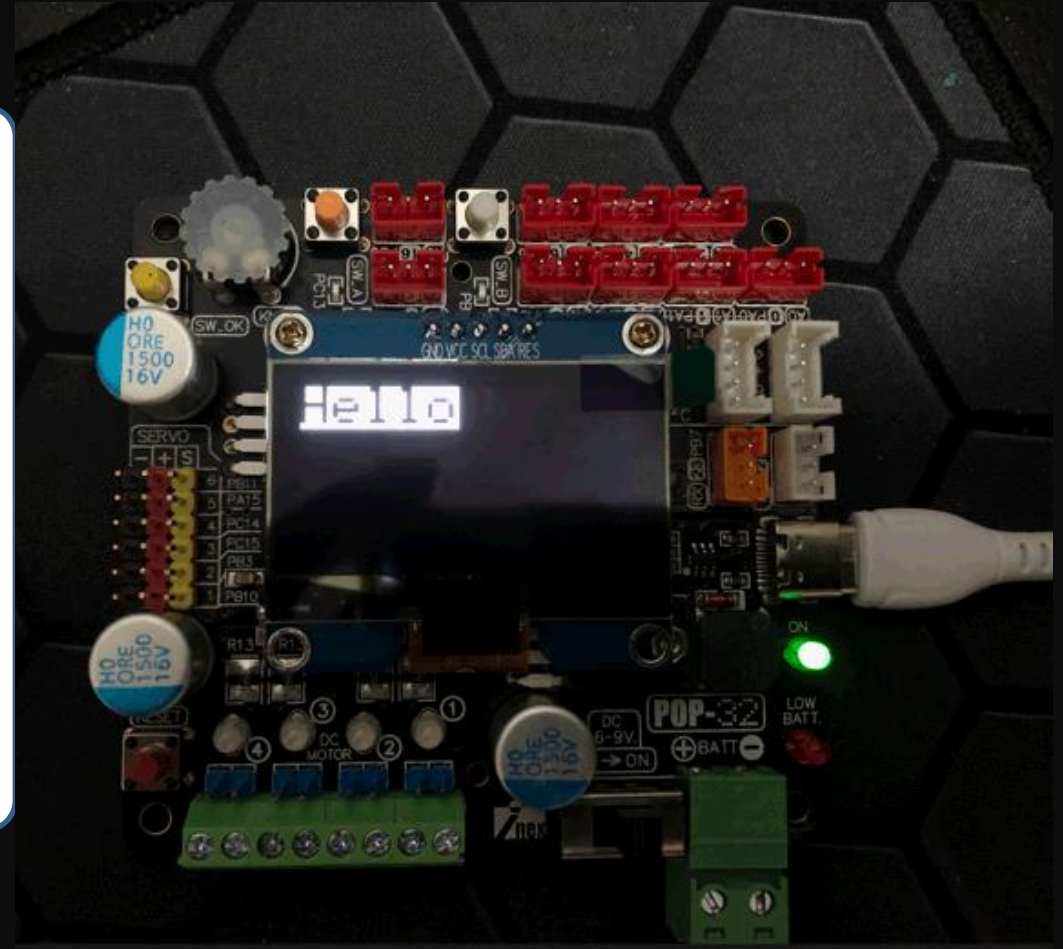


ตัวอักษรสีดำพื้นหลังตัวอักษรเป็นสีขาวเท่านั้น

ตัวอย่างกำหนดขนาดตัวอักษร

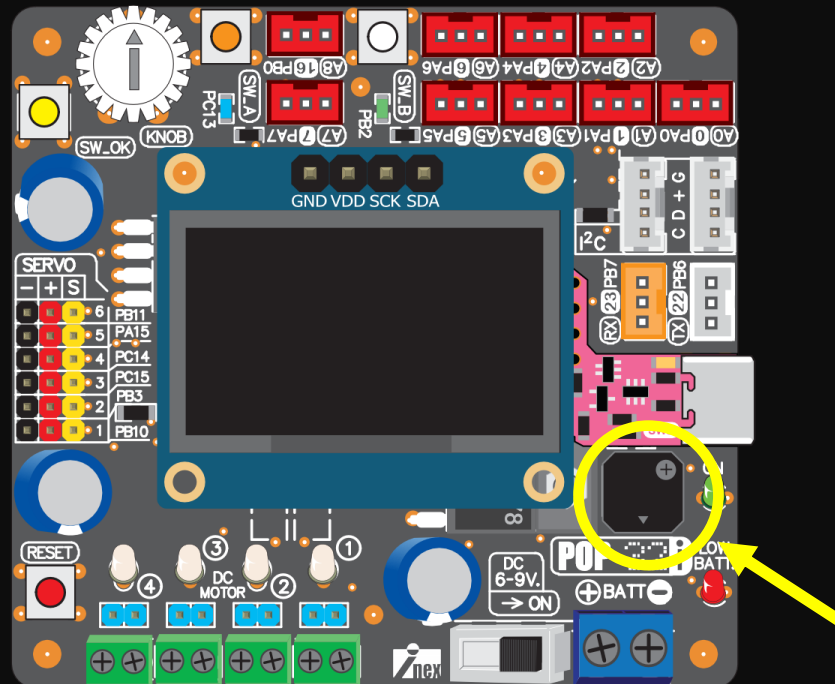
```
#include <POP32.h>
void setup(){
  oled.setTextSize(2);
  oled.setTextColor(BLACK,WHITE);
  oled.text(0,0,"Hello world");
  oled.show();
}
void loop(){}

```



ตัวอักษรสีดำพื้นหลังตัวอักษรเป็นสีขาวมีขนาดเป็น 2 เท่า

กลุ่มคำสั่งสร้างสัญญาณเสียงออกลำโพงเปียโซ



การสร้างเสียงออกลำโพง

beep()

sound(freq,time)

freq คือ ความถี่เสียง

time คือ เวลา 1/100 วินาที

มีค่าความถี่ย่าน 300 Hz ถึง 3000 Hz

ตัวอย่างโปรแกรมสร้างเสียงออกลำโพง

ตัวอย่างการใช้งานคำสั่ง beep

```
#include <POP32.h>
void setup(){}
void loop(){
    beep();
    delay(500);
}
```

สร้างเสียง 500 Hz นาน 0.1 วินาที ต่อเนื่อง

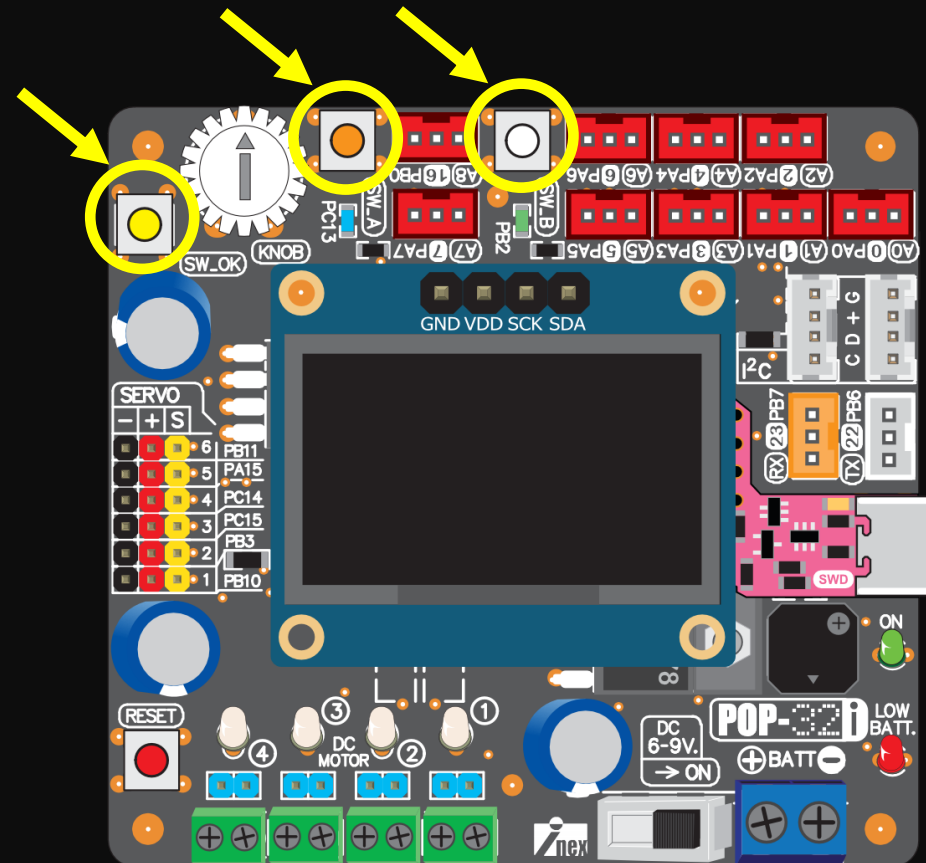
ตัวอย่างการใช้งานคำสั่ง sound

```
#include <POP32.h>
void setup(){}
void loop(){
    sound(1000,100);
    delay(500);
}
```

สร้างเสียง 1 kHz นาน 0.1 วินาที ต่อเนื่อง

กลุ่มคำสั่งอ่านค่าสถานะสวิตช์

A,B และ OK



คำสั่งอ่านค่าสถานะสวิตช์

ทำหน้าที่อ่านค่าสถานะการกดสวิตช์ A, B และ OK

รูปแบบ

SW_A()

SW_B()

SW_OK()

การคืนค่า

- คืนค่าเป็น true(1) ในกรณีที่สวิตช์ถูกกด
- คืนค่าเป็น false(0) ในกรณีที่สวิตช์ไม่ถูกกด

คำสั่งรอกการกดสวิตช์

ทำหน้าที่รอกจนกระทั่งสวิตช์ที่กำหนด

(LED ประจำตัวสวิตช์ จะกระพริบต่อเนื่องเพื่อแจ้งผู้ใช้งานในระหว่างรอกการกดสวิตช์)

รูปแบบ

waitSW_A()

waitSW_B()

waitSW_OK()

waitAnykey()

การคืนค่า

ไม่มีการคืนค่า

คำสั่งตรวจสอบเงื่อนไข if-else

```
if (เงื่อนไข) {  
    คำสั่ง1 ทำเมื่อเงื่อนไขเป็นจริง;  
    ...  
}  
else {  
    คำสั่ง2 ทำเมื่อเงื่อนไขเป็นเท็จ;  
    ...  
}
```

ตัวแปร Boolean

จริง : True : 1

เท็จ : False : 0

ตัวอย่างตรวจจบการกดสวิตช์ A

```
#include <POP32.h>
void setup(){}
void loop()
{
  if(SW_A())
  {
    beep();
    delay(200);
  }else{
    sound(2000,200);
    delay(200);
  }
}
```

หลังจากกดสวิตช์ A หรือ ไม่กดสวิตช์เสียงจะถูกกำเนิดความถี่ที่ต่างกัน

ตัวอย่างตรวจจบการกดสวิตช์ A และ B

```
#include <POP32.h>
void setup(){
}
void loop()
{
    if(SW_A())
    {
        beep();
        delay(200);
    }
    if(SW_B())
    {
        sound(2000,200);
        delay(200);
    }
}
```

หลังจากกดสวิตช์ A และ B เสียงจะถูกกำเนิดความถี่ที่ต่างกัน

ตัวอย่างการใช้ && (AND)

```
#include <POP32.h>
void setup(){
}
void loop()
{
    if(SW_A() && SW_B())
    {
        beep();
        delay(200);
    }else{
        sound(2000, 200);
        delay(200);
    }
}
```

กดสองปุ่มพร้อมกัน

ไม่กดสวิตช์

ตัวอย่างการใช้ || (OR)

```
#include <POP32.h>
void setup(){
}
void loop()
{
    if(SW_A() || SW_B())
    {
        beep();
        delay(200);
    }else{
        sound(2000, 200);
        delay(200);
    }
}
```

กดปุ่มใดปุ่มหนึ่ง

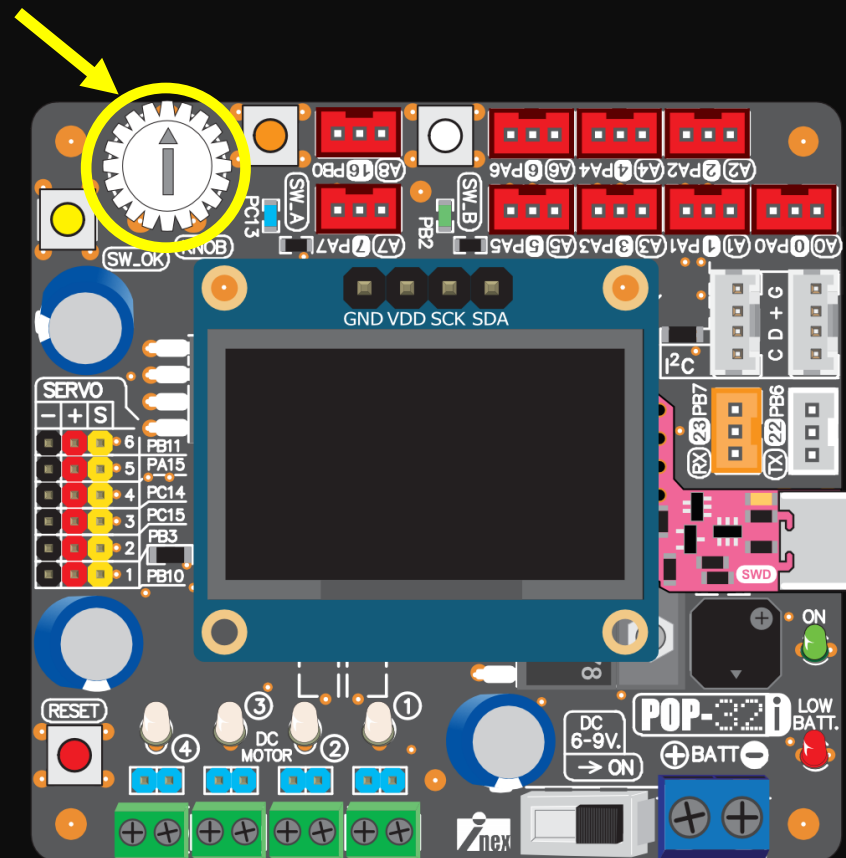
ไม่กดสวิตช์

ตัวอย่างรอกนกระทั่งมีการกดสวิตช์ OK

```
#include <POP32.h>
void setup(){
    oled.text(0,0,"Press SW_OK");
    oled.show();
    waitSW_OK();
}
void loop(){
    beep();
    delay(1000);
}
```

หลังจากกดสวิตช์ OK เสียงจะถูกกำเนิดทุกๆ 1 วินาที

กลุ่มคำสั่งอ่านค่า KNOB



คำสั่ง knob()

ทำหน้าที่อ่านค่าอะนาลอกจาก KNOB
ในช่วง 0 ถึง 4000

รูปแบบ

knob()

การคืนค่า

คืนค่าตั้งแต่ 0 ถึง 4000 จากการปรับค่า

คำสั่ง knob(x)

ทำหน้าที่อ่านค่าอะนาลอกจาก KNOB
ในช่วง 0 ถึง x ตามที่กำหนด

รูปแบบ

knob(x)

การคืนค่า

คืนค่าตั้งแต่ 0 ถึง x จากการปรับค่า

คำสั่ง knob(x,y)

ทำหน้าที่อ่านค่าอะนาลอกจาก KNOB ในช่วง x ถึง y ตามที่กำหนด

รูปแบบ

knob(x,y)

การคืนค่า

คืนค่าตั้งแต่ x ถึง y จากการปรับค่า

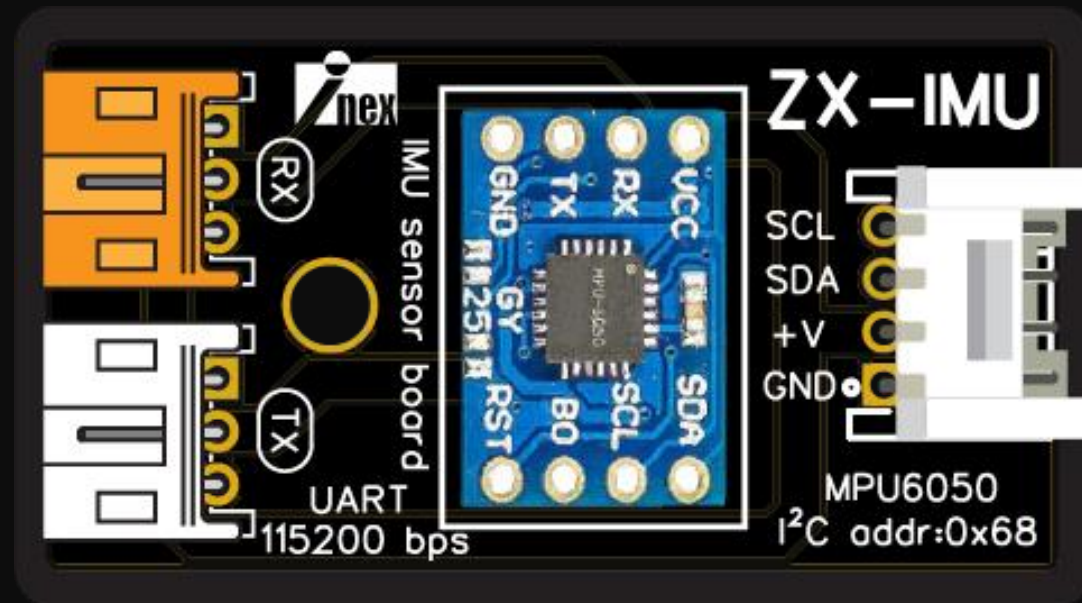
ตัวอย่างอ่านค่า knob

```
#include <POP32.h>
void setup() {
}
void loop() {
    oled.text(0,0,"knob=%d ",knob());
    oled.show();
}
```

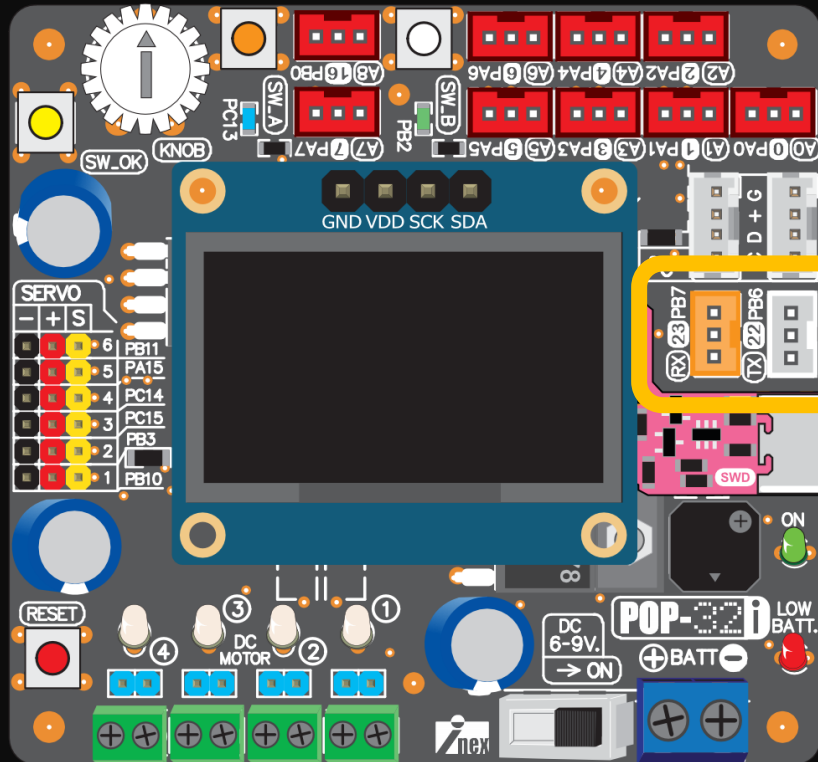
ทดสอบปรับหมุนที่ knob แล้วลอง
เปลี่ยนการอ่านค่าทั้ง 3 รูปแบบ

ZX-IMU

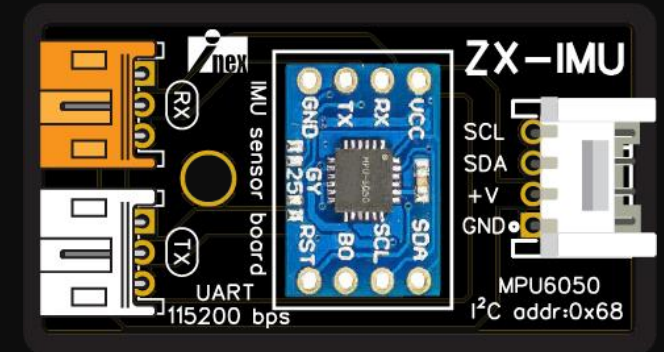
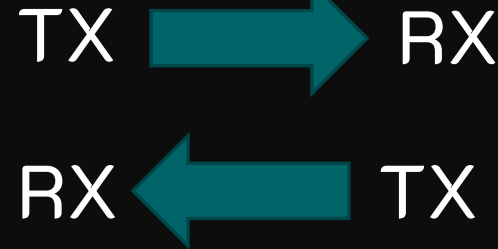
สามารถต่อใช้งานได้ทั้ง I2C(SDA,SCL) และ Serial(Rx,Tx) โมดูลประกอบไปด้วย MPU6050 เป็นเซนเซอร์ Gyro และ Accelerometer โดยมีไมโครคอนโทรลเลอร์ ใช้เป็นตัวกลางเพื่ออ่านค่าจาก MPU6050 และส่งค่าออก Serial(Rx,Tx)

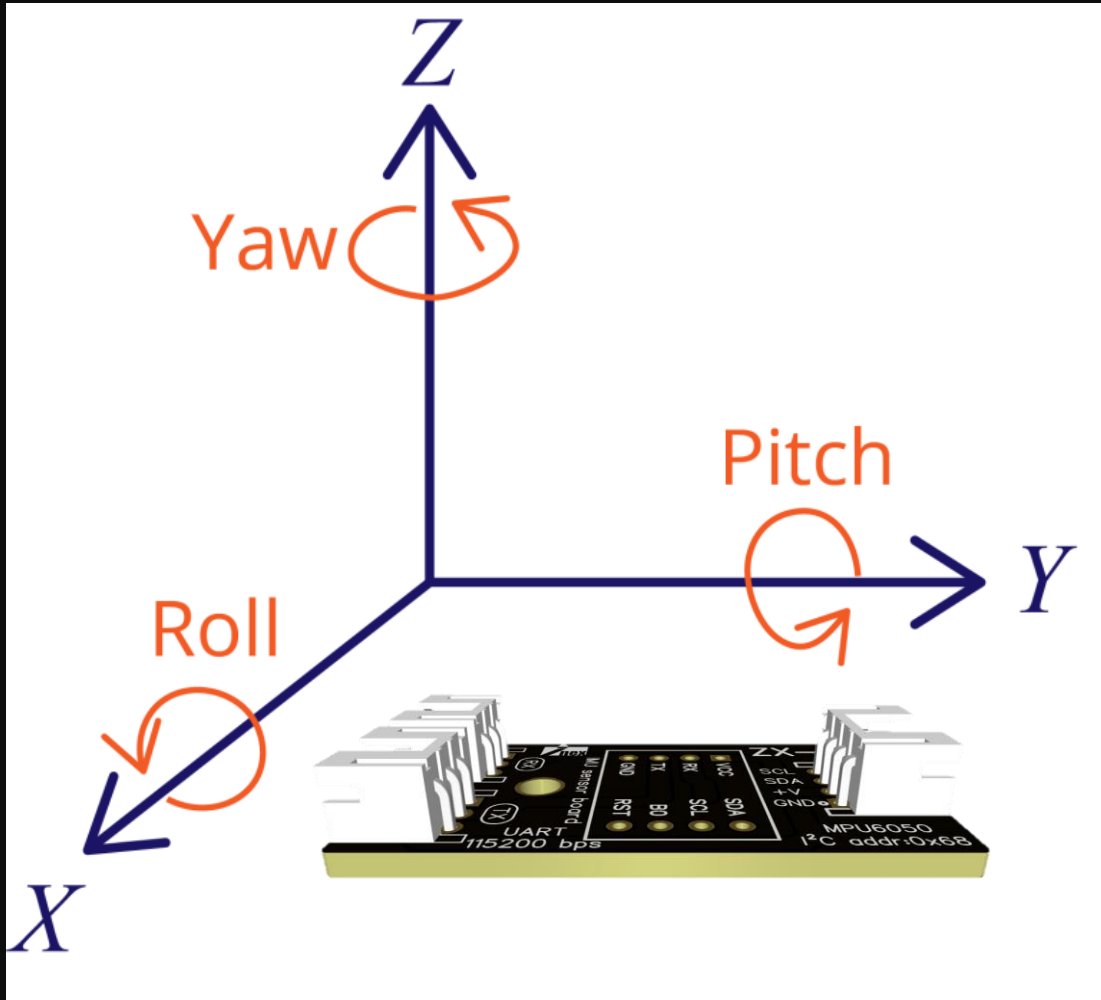


การต่อใช้งาน



ใช้แบบ UART





Yaw

นำมาใช้เป็นเข็มทิศอ้างอิงให้หุ่นยนต์

คำสั่ง

Serial.begin(baud rate)

กำหนดอัตราการสื่อสารข้อมูลเช่น 115200 bs

Serial.write(data)

ส่งข้อมูลออกไป ได้เพียงแค่ทีละ 1 byte

Serial.available()

จำนวนไบต์ที่สามารถอ่านได้

Serial.read()

อ่านข้อมูลอนุกรมที่เข้ามา

ฟังก์ชันรีเซ็ตมุม Yaw

ขณะรีเซ็ตหุ่นยนต์ต้องอยู่นิ่งๆ อย่างน้อย
3 วินาที ก่อนใช้งาน

เริ่มต้นใช้งาน Serial ช่องหมายเลข 1

ปรับปรุงค่ามุม pitch และ roll ให้เป็น 0

```
void zeroYaw() {  
    Serial1.begin(115200);delay(100);  
    Serial1.write(0XA5);Serial1.write(0X54);delay(100);  
    Serial1.write(0XA5);Serial1.write(0X55);delay(100);  
    Serial1.write(0XA5);Serial1.write(0X52);delay(100);  
}
```

เลือกโหมดอัตโนมัติ

ปรับปรุงค่ามุม Yaw ให้เป็น 0

ฟังก์ชันอ่านค่า Yaw

```
float pvYaw;
uint8_t rxCnt = 0, rxBuf[8];
bool getIMU() {
    while (Serial1.available()) {
        rxBuf[rxCnt] = Serial1.read();
        if (rxCnt == 0 && rxBuf[0] != 0xAA) return false;
        rxCnt++;
        if (rxCnt == 8) {
            rxCnt = 0;
            if (rxBuf[0] == 0xAA && rxBuf[7] == 0x55) {
                pvYaw = (int16_t)(rxBuf[1] << 8 | rxBuf[2]) / 100.f;
                return true;
            }
        }
    }
    return false;
}
```

เมื่อมีข้อมูลส่งเข้ามา

อ่านค่าเก็บไว้ในตัวแปร

ตรวจสอบความถูกต้อง

ตรวจสอบความถูกต้อง

อ่านและคำนวณค่ามุม Yaw

สร้างโปรแกรมอ่านค่าYaw

Code -> Test_ZX-IMU.ino

```
#include <POP32.h>
void setup() {
  zeroYaw();
}
void loop() {
  if (SW_A()) {
    zeroYaw();
    beep();
  }
  if (getIMU()) {
    oled.text(0, 0, "Yaw=%f", pvYaw);
    oled.show();
  }
}
```

กดปุ่ม A เพื่อรีเซ็ตเข็มทิศ
จากนั้นหมุนหุ่นยนต์เพื่อทดสอบ

